W2 — LLM Adapter: 6-Metric Canonical Rubric

Scope: Rewrite `parseModelOutput()` and `LLMResponse` types in `backend/src/services/llm-adapter.ts` to accept the canonical 6-metric schema emitted by the W1-rewritten judge prompt. Coordinate with W3 (claim_scores column rename) and judge-worker write-back.

Canonical contract (from W1):

```
{ "fc": 0.00-1.00, "ca": 0.00-1.00, "hr": 0.00-1.00, "citAcc": 0.00-1.00,
  "rel": 0.00-1.00, "comp": 0.00-1.00,
  "verdict": "APPROVE"|"FLAG"|"UNCERTAIN", "confidence": 0.0-1.0,
  "reason": "string (<=1200 chars)",
  "evidence": [{"url": "string", "tier": "gov"|"state_media"|"advisory"|"other"}] }
```

Note: per PRD v5.09 the judge emits **six** metrics (fc, ca, hr, citAcc, rel, comp). `ca` and `citAcc` are distinct (ca = citation coverage, citAcc = citation accuracy) — matches the W1 prompt rewrite.

---

1. `backend/src/services/llm-adapter.ts` — unified diff

```
@@ -15,21 +15,50 @@
 // ---------------------------------------------------------------------------
 // Public types
 // ---------------------------------------------------------------------------

-export interface LLMResponse {
+/** Canonical 6-metric rubric, all floats in [0,1] (step 0.05 tolerated). */
+export interface JudgeRubric {
+  fc: number;      // factual correctness
+  ca: number;      // citation coverage
+  hr: number;      // hallucination risk (LLM emits inverted: 1=safe, 0=hallucinated)
+  citAcc: number;  // citation accuracy
+  rel: number;     // relevance
+  comp: number;    // completeness
+}
+
+export type EvidenceTier = 'gov' | 'state_media' | 'advisory' | 'other';
+
+export interface EvidenceItem {
+  url: string;
+  tier: EvidenceTier;
+}
+
+/** Judge run result — renamed conceptually from LLMResponse.  */
+export interface JudgeRunResult {
   verdict: 'APPROVE' | 'FLAG' | 'UNCERTAIN';
   confidence: number;
-  rubric: Record<string, number>;
```

```
+  rubric: JudgeRubric;
+  evidence: EvidenceItem[];
   reason: string;
   rawResponse: string;
   tokensPrompt: number;
   tokensCompletion: number;
   costCents: number;
   latencyMs: number;
 }
+
+/** Backwards-compat alias — kept so downstream imports don't churn. */
+export type LLMResponse = JudgeRunResult;
+
+const METRIC_KEYS = ['fc', 'ca', 'hr', 'citAcc', 'rel', 'comp'] as const;
+type MetricKey = typeof METRIC_KEYS[number];
+const EVIDENCE_TIERS: ReadonlySet<EvidenceTier> = new Set([
+  'gov', 'state_media', 'advisory', 'other',
+]);

@@ -79,58 +108,93 @@
-/**
- * Parse the model's text output into a structured LLMResponse.
- * Falls back to verdict = 'UNCERTAIN', confidence = 0 if JSON is absent or malformed.
- */
+/** Clamp v to [0,1]; non-numbers → 0. */
+function clamp01(v: unknown): number {
+  if (typeof v !== 'number' || !Number.isFinite(v)) return 0;
+  return Math.min(1, Math.max(0, v));
+}
+
+/**
+ * Parse the model's text output into a structured JudgeRunResult.
+ *
+ * Behaviour:
+ *   - JSON extraction: first {...} block in raw string (preamble tolerant).
+ *   - Each of the 6 metrics is clamped to [0,1]; missing/invalid → 0.
+ *   - If ANY metric is missing, verdict is downgraded to UNCERTAIN
+ *     (the gate already treats UNCERTAIN as FLAGGED — safe default).
+ *   - hr is already emitted 0–1 by the LLM (higher = safer); no inversion here.
+ *   - evidence[] entries missing url or unknown tier are dropped.
+ *   - Malformed JSON or missing verdict → UNCERTAIN with zero rubric.
+ */
 function parseModelOutput(
   raw: string,
   tokensPrompt: number,
   tokensCompletion: number,
   latencyMs: number,
-): LLMResponse {
+): JudgeRunResult {
   const jsonMatch = raw.match(/\{[\s\S]*\}/);
```

```
   if (!jsonMatch) return makeUncertain(raw, tokensPrompt, tokensCompletion, latencyMs);

   let parsed: unknown;
   try { parsed = JSON.parse(jsonMatch[0]); }
   catch { return makeUncertain(raw, tokensPrompt, tokensCompletion, latencyMs); }

   const obj = parsed as Record<string, unknown>;
-  const verdict = obj['verdict'];
-  if (verdict !== 'APPROVE' && verdict !== 'FLAG') {
+  const rawVerdict = obj['verdict'];
+  if (rawVerdict !== 'APPROVE' && rawVerdict !== 'FLAG' && rawVerdict !== 'UNCERTAIN') {
     return makeUncertain(raw, tokensPrompt, tokensCompletion, latencyMs);
   }
+  let verdict: JudgeRunResult['verdict'] = rawVerdict;

   const confidence = clamp01(obj['confidence']);

-  const rubricRaw = (obj['rubric'] ?? {}) as Record<string, unknown>;
-  const rubric: Record<string, number> = {};
-  for (const [k, v] of Object.entries(rubricRaw)) {
-    rubric[k] = typeof v === 'number' ? Math.round(Math.min(5, Math.max(1, v))) : 0;
+  // Metrics live at the top level (W1 prompt). Fall back to obj.rubric
+  // for backward compatibility with the old inner-rubric shape.
+  const src = (obj['rubric'] && typeof obj['rubric'] === 'object')
+    ? (obj['rubric'] as Record<string, unknown>)
+    : obj;
+  const rubric: JudgeRubric = {
+    fc: 0, ca: 0, hr: 0, citAcc: 0, rel: 0, comp: 0,
+  };
+  let missingAny = false;
+  for (const k of METRIC_KEYS) {
+    if (src[k] === undefined || src[k] === null) missingAny = true;
+    rubric[k as MetricKey] = clamp01(src[k]);
+  }
+  if (missingAny && verdict !== 'UNCERTAIN') verdict = 'UNCERTAIN';
+
+  const evRaw = Array.isArray(obj['evidence']) ? obj['evidence'] : [];
+  const evidence: EvidenceItem[] = [];
+  for (const e of evRaw) {
+    if (!e || typeof e !== 'object') continue;
+    const er = e as Record<string, unknown>;
+    const url = typeof er['url'] === 'string' ? er['url'] : '';
+    const tier = er['tier'];
+    if (!url || typeof tier !== 'string' || !EVIDENCE_TIERS.has(tier as EvidenceTier)) continue;
+    evidence.push({ url, tier: tier as EvidenceTier });
   }

   const reason = typeof obj['reason'] === 'string'
-    ? obj['reason'].slice(0, 600)
+    ? obj['reason'].slice(0, 1200)
```

```
     : '';

   const costCents = Math.round(((tokensPrompt + tokensCompletion) / 1000) * 0.2);

-  return { verdict, confidence, rubric, reason, rawResponse: raw,
+  return { verdict, confidence, rubric, evidence, reason, rawResponse: raw,
            tokensPrompt, tokensCompletion, costCents, latencyMs };
 }

-function makeUncertain(raw, tokensPrompt, tokensCompletion, latencyMs): LLMResponse {
+function makeUncertain(
+  raw: string, tokensPrompt: number, tokensCompletion: number, latencyMs: number,
+): JudgeRunResult {
   return {
     verdict: 'UNCERTAIN',
     confidence: 0,
-    rubric: {},
+    rubric: { fc: 0, ca: 0, hr: 0, citAcc: 0, rel: 0, comp: 0 },
+    evidence: [],
     reason: 'Failed to parse model output as valid JSON verdict.',
     rawResponse: raw,
     tokensPrompt, tokensCompletion,
     costCents: 0,
     latencyMs,
   };
 }
```

Not changed: `callOpenAICompat`, `callAnthropic`, `callGoogle`, `callProvider`, `resolveProvider`, cost estimate arithmetic. The `response_format: { type: 'json_object' }` + W1 prompt instructions keep the top-level JSON shape intact across all providers.

---

2. `backend/src/workers/judge-worker.ts` — unified diff

```
@@ -28,7 +28,7 @@
 import {
   callProvider,
   resolveProvider,
   type ProviderConfig,
-  type LLMResponse,
+  type JudgeRunResult,
+  type JudgeRubric,
 } from '../services/llm-adapter.js';

@@ -282,59 +282,58 @@
 // ---------------------------------------------------------------------------
-// Judge rubric → claim_scores write-back (US-A007)
+// Judge rubric → claim_scores write-back (US-A007, 6-metric canonical)
 // ---------------------------------------------------------------------------
```

```
-/** Rubric keys emitted by the judge prompt mapped to metric_code enum values. */
-const RUBRIC_KEY_MAP: Record<string, string> = {
-  FC: 'FC', fc: 'FC',
-  CitAcc: 'CitAcc', citacc: 'CitAcc', citationAccuracy: 'CitAcc', CA: 'CitAcc',
-  HR: 'HR', hr: 'HR', hallucinationRisk: 'HR',
-  Rel: 'Rel', rel: 'Rel', relevance: 'Rel',
-  Comp: 'Comp', comp: 'Comp', completeness: 'Comp',
-};
+/**
+ * Map canonical rubric keys → metric_code enum. W3 will add `CA` to the
+ * enum if it is not already present; coordinate the migration there.
+ * Column layout on claim_scores: (fc, ca, hr, cit_acc, rel, comp) [W3].
+ */
+const METRIC_CODE: Record<keyof JudgeRubric, string> = {
+  fc: 'FC', ca: 'CA', hr: 'HR', citAcc: 'CitAcc', rel: 'Rel', comp: 'Comp',
+};

 function snapToGrid(v: number): number {
   return Math.max(0, Math.min(1, Math.round(v * 20) / 20));
 }

 async function persistJudgeScores(
   claimId: string,
-  rubric: Record<string, number>,
+  rubric: JudgeRubric,
   log: WorkerLogger,
 ): Promise<void> {
-  const entries = Object.entries(rubric)
-    .map(([k, v]) => ({ metric: RUBRIC_KEY_MAP[k], value: snapToGrid(v) }))
-    .filter((e): e is { metric: string; value: number } => Boolean(e.metric));
-
-  if (entries.length === 0) return;
+  const entries = (Object.entries(rubric) as Array<[keyof JudgeRubric, number]>)
+    .map(([k, v]) => ({ metric: METRIC_CODE[k], value: snapToGrid(v) }));

   try {
     for (const { metric, value } of entries) {
       await sql`
         INSERT INTO claim_scores (
           claim_id, metric, value, weight, inverted,
           scored_by, scored_by_role, is_original
         ) VALUES (
           ${claimId}::uuid,
           ${metric}::metric_code,
           ${value},
           1.000,
-          FALSE,
+          ${metric === 'HR'},  -- HR stored as inverted flag per W3 schema
           ${SYSTEM_ACTOR_ID}::uuid,
           'LLM',
```

```
          TRUE
        )
        ON CONFLICT (claim_id, metric, scored_by_role)
        DO UPDATE SET value = EXCLUDED.value,
                      scored_by = EXCLUDED.scored_by,
                      created_at = now()
      `;
    }
  } catch (err) {
    log.warn({ err, claimId }, 'judge-worker: claim_scores write-back failed (non-fatal)');
  }
}

+/** Weighted aggregate stored in llm_runs.score and claims.stage1_score. */
+function aggregateScore(r: JudgeRubric): number {
+  return (r.fc + r.ca + r.hr + r.citAcc + r.rel + r.comp) / 6;
+}

@@ -347,14 +346,14 @@
 // Confidence gate → next claim status
 // ---------------------------------------------------------------------------

 function nextStatus(
   confidence: number,
-  verdict: LLMResponse['verdict'],
+  verdict: JudgeRunResult['verdict'],
 ): 'AUTO_APPROVED' | 'FLAGGED' | 'PENDING_CONSENSUS' {
   if (verdict === 'UNCERTAIN') return 'FLAGGED';
   if (confidence >= 0.90 && verdict === 'APPROVE') return 'AUTO_APPROVED';
   if (confidence < 0.50) return 'FLAGGED';
   return 'PENDING_CONSENSUS';
 }

@@ -451,7 +450,7 @@
-  let response: LLMResponse | null = null;
+  let response: JudgeRunResult | null = null;

@@ -515,23 +514,26 @@
-  // 5. Persist llm_run
+  // 5. Persist llm_run — score column holds weighted aggregate of 6 metrics
+  const aggregate = aggregateScore(response.rubric);
   await persistLlmRun({
     claimId: claim.id,
     providerId: usedProvider.id,
     providerSnapshot: { … },
     promptHash,
-    response,
+    response: { ...response, confidence: response.confidence }, // unchanged
     fallbackHistory,
+    aggregate,
```

```
   });

   await persistJudgeScores(claim.id, response.rubric, log);

-  const to = nextStatus(response.confidence, response.verdict);
-  await transitionClaim(claim.id, to, response.confidence, log);
+  const to = nextStatus(response.confidence, response.verdict);
+  // stage1_score = aggregate; confidence-gate thresholds unchanged.
+  await transitionClaim(claim.id, to, aggregate, log);
```

`persistLlmRun` signature change: add `aggregate: number`; change the INSERT to write `${aggregate}` into `score` instead of `${response.confidence}`. (The `confidence` value is still preserved as part of the rubric/reason JSON if a `runs_meta` column is added in W3; otherwise it lives only in `raw_response` until an explicit column is added.)

```
 async function persistLlmRun(params: {
   claimId: string;
   providerId: string;
   providerSnapshot: Record<string, unknown>;
   promptHash: string;
-  response: LLMResponse;
+  response: JudgeRunResult;
+  aggregate: number;
   fallbackHistory: Array<{ provider: string; error: string; ts: number }>;
 }): Promise<void> {
-  ... VALUES (..., ${response.confidence}, ${response.verdict}, ...)
+  ... VALUES (..., ${params.aggregate}, ${response.verdict}, ...)
```

---

3. Confidence gate (unchanged thresholds)

- Gate reads `response.confidence` (LLM-reported, 0–1). Thresholds stay **≥0.90** → **AUTO_APPROVED**, **<0.50** → **FLAGGED**, else PENDING_CONSENSUS.
- `UNCERTAIN` still collapses to FLAGGED.
- NEW: `llm_runs.score` and `claims.stage1_score` now store `aggregate = mean(fc, ca, hr, citAcc, rel, comp)` rather than the raw confidence. This is the dashboard-facing quality metric; confidence stays the gate input.

---

4. Import / type reference audit

Grep performed: `JudgeRunResult|JudgeRubric` — currently zero hits (new types). `LLMResponse` is the existing import.

```
Grep LLMResponse --type ts:
  backend/src/services/llm-adapter.ts   (definition)
  backend/src/workers/judge-worker.ts   (import, variable type)
```

Downstream diffs required:

File	Change
`backend/src/services/llm-adapter.ts`	Full diff §1.
`backend/src/workers/judge-worker.ts`	Full diff §2.
`backend/test/llm-adapter.parse.spec.ts`	NEW — see §5.
`backend/test/judge-worker.*.spec.ts` (if present)	Update any fixture constructing a fake `LLMResponse` — must supply the new `rubric: JudgeRubric` shape with 6 floats and `evidence: []`.
Any FE type mirrors (`src/types/*.ts`)	None found for rubric; the FE receives rubric only via API responses from W3. Skip here.

Run before implementation:

```
rg --type ts "LLMResponse|rubric:" backend/src backend/test
rg --type ts "stage1_score|claim_scores|llm_runs" backend/src
```

Any test fixture literal `{ verdict, confidence, rubric: {} }` must become `{ verdict, confidence, rubric: { fc:0, ca:0, hr:0, citAcc:0, rel:0, comp:0 }, evidence: [] }`.

---

5. Vitest stub — `backend/test/llm-adapter.parse.spec.ts`

```
import { describe, it, expect } from 'vitest';
// parseModelOutput is not exported — expose via __test__ namespace or
// re-export behind `if (process.env.NODE_ENV === 'test')`. Simplest:
// change llm-adapter.ts to `export function parseModelOutput(...)`.
import { parseModelOutput } from '../src/services/llm-adapter.js';

const base = { tokensPrompt: 10, tokensCompletion: 20, latencyMs: 5 };

describe('parseModelOutput (6-metric canonical)', () => {
  it('parses a valid 6-metric APPROVE payload', () => {
    const raw = JSON.stringify({
      fc: 0.95, ca: 0.9, hr: 0.85, citAcc: 0.8, rel: 0.9, comp: 0.7,
      verdict: 'APPROVE', confidence: 0.93, reason: 'ok',
      evidence: [{ url: 'https://gov.vn/x', tier: 'gov' }],
    });
    const r = parseModelOutput(raw, base.tokensPrompt, base.tokensCompletion, base.latencyMs);
    expect(r.verdict).toBe('APPROVE');
    expect(r.confidence).toBeCloseTo(0.93);
    expect(r.rubric).toEqual({
      fc: 0.95, ca: 0.9, hr: 0.85, citAcc: 0.8, rel: 0.9, comp: 0.7,
    });
    expect(r.evidence).toHaveLength(1);
    expect(r.evidence[0].tier).toBe('gov');
  });

  it('downgrades verdict to UNCERTAIN when a metric is missing', () => {
```

```
    const raw = JSON.stringify({
      fc: 0.9, ca: 0.9, hr: 0.9, citAcc: 0.9, rel: 0.9, /* comp missing */
      verdict: 'APPROVE', confidence: 0.95, reason: 'x',
    });
    const r = parseModelOutput(raw, 1, 1, 1);
    expect(r.verdict).toBe('UNCERTAIN');
    expect(r.rubric.comp).toBe(0);
  });

  it('clamps overflow values into [0,1]', () => {
    const raw = JSON.stringify({
      fc: 1.7, ca: -0.3, hr: 0.5, citAcc: 0.5, rel: 0.5, comp: 0.5,
      verdict: 'FLAG', confidence: 2.0, reason: 'x',
    });
    const r = parseModelOutput(raw, 1, 1, 1);
    expect(r.rubric.fc).toBe(1);
    expect(r.rubric.ca).toBe(0);
    expect(r.confidence).toBe(1);
  });

  it('falls back to UNCERTAIN on malformed JSON', () => {
    const r = parseModelOutput('not json {{{', 1, 1, 1);
    expect(r.verdict).toBe('UNCERTAIN');
    expect(r.confidence).toBe(0);
    expect(r.rubric).toEqual({ fc: 0, ca: 0, hr: 0, citAcc: 0, rel: 0, comp: 0 });
    expect(r.evidence).toEqual([]);
  });

  it('drops evidence items with unknown tier or missing url', () => {
    const raw = JSON.stringify({
      fc: 1, ca: 1, hr: 1, citAcc: 1, rel: 1, comp: 1,
      verdict: 'APPROVE', confidence: 0.9, reason: '',
      evidence: [
        { url: 'https://a', tier: 'gov' },
        { url: 'https://b', tier: 'weird' },
        { tier: 'gov' },
      ],
    });
    const r = parseModelOutput(raw, 1, 1, 1);
    expect(r.evidence).toEqual([{ url: 'https://a', tier: 'gov' }]);
  });
});
```

Prerequisite: change `function parseModelOutput` → `export function parseModelOutput` in llm-adapter.ts (already in diff §1 implicitly; add the `export` keyword). No other test scaffolding needed — vitest already wired in `backend/package.json`.

---

6. Coordination notes (W1/W3 handoff)

- **W1 (prompt)**: must emit the 6 keys at the top level (not nested in `rubric`) and `hr` as 0–1 where **1 = no hallucination**. Parser still accepts `{rubric:{...}}` shape for transitional safety.
- **W3 (schema)**: claim_scores must accept `CA` as a metric_code enum value. If the enum already has all six, nothing to migrate; otherwise a forward-only migration in `backend/migrations/` and `supabase/migrations/` is needed. `llm_runs.score` column semantics change from "confidence" to "aggregate" — document in W3 changelog.
- **No FE contract change** yet; rubric shape is only exposed to FE via `GET /v1/claims/:id` payloads shaped by W3.

---

7. Risk / rollback

- Single risk: if W1 prompt rewrite is not deployed before this parser, every judge call will yield `UNCERTAIN` (missing-field path) and all claims will flip to FLAGGED. Mitigation: keep the legacy `rubric:{}` inner-shape fallback in the parser (included in §1 diff), ship W1 + W2 in the same release.
- Rollback: revert both files; the old 5-metric enum values remain in DB and are simply not written until revert.